Addis Ababa University — School of Information Technology and Engineering
Software Testing and Validation — Final Project

# Defect Log and Metrics

Clinic Booking System

| | |
|---|---|
| **Student** | Biruk Mulatu |
| **Student ID** | [STUDENT ID — fill in before submission] |
| **Instructor** | Abel Tadesse |
| **Date** | 4 September 2026 |

## 1. Defect log

Three real defects were found and fixed during development — all through the project's own automated testing effort, none reported by an external user (there being none, this being a course project). Each entry below follows the lifecycle New → In Progress → Fixed → Verified → Closed. Severity is impact if shipped as-is; priority is urgency relative to other work.

### DEF-001 — Integration tests fail intermittently with a unique-constraint violation

**Severity:** Medium · **Priority:** High · **Status:** Closed · **Found:** Phase 6, first run of `AppointmentServiceIT` · **Component:** `integration/AppointmentServiceIT.java`

**Steps to reproduce:** Write `AppointmentServiceIT` as a plain `@SpringBootTest` (no `@Transactional`) with a `@BeforeEach` that inserts a `Patient` with a fixed username. Run `mvn clean verify`.

**Expected:** All test methods in the class pass independently.

**Actual:** The first method passes; every subsequent method fails:

```
org.springframework.dao.DataIntegrityViolationException: ... Unique index
or primary key violation: "PUBLIC.CONSTRAINT_INDEX_F ON
PUBLIC.PATIENT(USERNAME NULLS FIRST) VALUES ( /* 1 */ 'tigist' )"
```

**Root cause:** `@SpringBootTest` reuses one application context (and one H2 instance, via `DB_CLOSE_DELAY=-1`) across every method in the class. Without `@Transactional`, writes commit immediately and are never rolled back, so the second `@BeforeEach` collides with the first test's committed row.

**Fix:** Added class-level `@Transactional` to `AppointmentServiceIT` and `AppointmentJourneyIT`, so each method's writes roll back at the end of the method. Deliberately *not* applied to the Selenium `BookingJourneyIT`: Selenium drives the app over a real HTTP socket from a separate thread, so an open test-thread transaction would be invisible to the server thread — that class avoids the collision with a distinct seeded username per test instead.

**Verification:** `mvn clean verify` passes with all methods green, run repeatedly.

---

### DEF-002 — CI coverage-summary step fails with a confusing secondary error whenever the build already failed

**Severity:** Low · **Priority:** Medium · **Status:** Closed · **Found:** during the deliberate regression demo (§3) · **Component:** `.github/workflows/ci.yml`, "Publish coverage summary" step

**Steps to reproduce:** Break a test in `et.aau.clinic.core` so `mvn clean verify` fails during the unit-test phase; push to a branch and let the GitHub Actions `CI` workflow run.

**Expected:** Only the real test failure is reported; the coverage-summary step is skipped or clearly reports "no coverage available".

**Actual:** The "Publish coverage summary" step (bound with `if: always()`) also fails, burying the real failure:

```
awk: fatal: cannot open file `target/site/jacoco/jacoco.csv' for reading: No such file or directory
Error: Process completed with exit code 2.
```

**Root cause:** `jacoco.csv` is produced by a plugin goal bound to Maven's `verify` phase. Maven's fail-fast behaviour stops the build at the first failing phase, so when unit tests fail in the earlier `test` phase, `verify` never runs and the CSV is never written — but the summary step assumed it always would be.

**Fix:** Added a file-existence check at the top of the step: if `jacoco.csv` is missing, write a one-line "no coverage report was generated" note to the job summary and exit 0 instead of trying to parse a file that was never created.

**Verification:** Reproduced on the `regression-demo` branch (failing run 33305001970), fixed in the same commit that reverted the regression (passing run 33305212152) — see §3.

---

### DEF-003 — Selenium system test is flaky under constrained CPU (Jenkins-in-Docker)

**Severity:** Medium · **Priority:** High · **Status:** Closed · **Found:** first real Jenkins pipeline run against `docker/jenkins/` · **Component:** `system/pages/*.java` (Page Objects)

**Steps to reproduce:** Run `BookingJourneyIT` in an environment with limited CPU relative to a GitHub-hosted runner (the Jenkins Docker container on a shared host). The page objects called `driver.findElement(...)` directly, immediately after a `.click()` that triggers a server-side redirect.

**Expected:** The system test passes reliably regardless of server response speed.

**Actual:**

```
org.openqa.selenium.NoSuchElementException: no such element: Unable to
locate element: {"method":"css selector","selector":"#confirmation-status"}
```

The same test passed reliably in GitHub Actions (hosted runner, more CPU) but failed the first time it ran inside the Jenkins container.

**Root cause:** Selenium's WebDriver spec only guarantees `click()` blocks until a triggered navigation *starts*, not until the resulting page has finished rendering. The page objects implicitly assumed the next page was already rendered — a race condition that held on a fast, lightly-loaded runner and broke under the container's tighter CPU budget.

**Fix:** Added `AbstractPage`, a common base class for all five page objects, with a `find(...)` helper backed by `WebDriverWait` + `ExpectedConditions.presenceOfElementLocated` (10s timeout). Every locator call now waits for its target element instead of assuming it is already there.

**Verification:** Reproduced in the Jenkins container (build #3 of `clinic-booking-pipeline`), fixed and verified locally, then re-run in the same container (build #4: `Finished: SUCCESS`) to confirm under the same constrained conditions that first exposed it.

The fix reduces the failure rate rather than proving the race eliminated - an explicit wait shortens the window, it does not remove it. As a data point: the same failure family recurred once on 2026-09-05 on a plain local `mvn clean verify` rerun under heavier-than-usual load, and was gone on immediate retry. Treated as mitigated, not closed against recurrence under sufficiently adverse timing.

---

## 2. Metrics

| Metric | Value | Interpretation |
|---|---|---|
| Defect density | 3 defects / 2.19 KLOC delivered (1,042 lines production code + 1,150 lines test code) ≈ **1.4 defects/KLOC** | Notably, **zero** of the 3 defects were in the 1,042 lines of application code itself (`domain`, `core`, `service`, `web`, `repository`) — all three were in test infrastructure (DEF-001, DEF-003) or CI scripting (DEF-002). Read against application code alone, density is 0/KLOC; the non-zero figure reflects how much of a small project's real defect risk lives in the harness that tests it, not just the thing being tested. |
| Defect removal efficiency (DRE) | 3 found internally / (3 found + 0 escaped) = **100%** | All three defects were caught by the project's own automated suite or its CI runs, before any external party (grader, user) ever exercised the system. This figure is trivially 100% only because the project has not yet passed through an external evaluation gate — see the residual-risk discussion in the test summary report for why it should not be over-read as "zero remaining risk". |
| Defects found vs. escaped | **3 found, 0 escaped** (as of 2026-09-04, pre-submission) | "Escaped" is not yet a meaningful category for a project that has not been graded — this row will only become informative if the instructor's evaluation surfaces further issues. |
| Branch coverage — `et.aau.clinic.core` (the graded target) | **100%** (34/34 branches), against an 80% gate enforced by `jacoco:check` | Every branch in the three pure business-rule classes (`FeeCalculator`, `BookingPolicy`, `AppointmentStateMachine`) is exercised — a direct consequence of deriving tests systematically from EP/BVA/decision-table/state-transition analysis (Part B) rather than writing tests ad hoc. |

| | | |
|---|---|---|
| Branch coverage — whole application (excluding `domain`, `config`, `ClinicApplication`, per the JaCoCo exclusions in `pom.xml`) | **70.6%** (48/68 branches); instruction coverage 60.5% (682/1128), line coverage 66.1% (158/239) — recomputed from a fresh `mvn clean verify` run on 2026-09-05, analysing 27 classes | The gate only requires `core/`, which remains untouched at 100%. The drop in this whole-application row from an earlier 100%/≈90%/≈89% is entirely explained by the JSON API added under `web.api` for the separate React frontend (`BookingApiController`, `SlotApiController`, `AppointmentApiController`, `AuthApiController`, and their DTOs): that package has no dedicated tests, so every one of its 20 branches and most of its instructions are unexercised. It is additive, ungraded code (see `README.md`), not a regression in the graded application's own coverage — the pre-existing `web/` and `service/` packages this row previously described are unchanged and still fully branch-covered. See §3 of the test summary report for what is deliberately left uncovered and why. |

## 3. Regression demonstration (context for the defect/CI story)

A deliberate regression was introduced to prove the pipeline actually catches breakage, not just that tests exist. Branch `regression-demo`, commit `9b6c9fe`, shifted Rule 1's CHILD/ADULT boundary from `age ≤ 17` to `age ≤ 16` — a one-line off-by-one that would silently overcharge every 17-year-old patient (ADULT fee, 250 ETB, instead of CHILD, 100 ETB) if it reached production.

| | Commit | CI run | Result |
|---|---|---|---|
| Before (regression introduced) | `9b6c9fe` | run 33305001970 | **Failure** (22s) — caught by `fee_atUpperBoundaryOfChildBand_age17_is100`, written in Phase 3, before this regression existed |
| After (regression reverted + DEF-002 fixed) | `4a98ab9` | run 33305212152 | **Success** (52s) — full suite green (58 unit + 9 integration + 2 system), coverage gate passed |

Full narrative in `docs/regression-demo.md`. This is exactly what boundary value analysis is for: the test exists at age 17 specifically because that is the boundary, and a one-line shift of the boundary was caught immediately by the one test written to sit on it.